Systèmes de numération et codage de l'information

Informatique — Licence 1 (L1)

Cours INF-L1-19

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Compter comme une machine : convertir les nombres entre les bases 2, 8, 10 et 16, calculer en binaire, coder les entiers négatifs en complément à deux, puis coder le texte, les images, le son — et détecter les erreurs de transmission.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Base 10, base 2, base 8 et base 16 : les conversions
• Leçon 2 — Opérations arithmétiques en binaire
• Leçon 3 — Représentation des nombres négatifs : le complément à deux
• Leçon 4 — Codage des caractères : ASCII et Unicode
• Leçon 5 — Codage des images et du son : initiation
• Leçon 6 — Codes détecteurs d'erreurs : le bit de parité
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Chiffres, lettres, photos, musique, vidéos : pour nous, ces informations n'ont rien en commun. Pour un ordinateur, ce sont exactement la même chose — de longues suites de 0 et de 1, les bits. Les circuits électroniques ne connaissent en effet que deux états stables, courant qui passe ou courant coupé, et toute l'informatique repose sur cette contrainte. Le codage de l'information est l'art de traduire le monde en suites de bits, puis de relire ces suites pour retrouver le nombre, le texte, l'image ou le son d'origine. Ce cours en pose les fondations, qui servent ensuite partout : architecture des ordinateurs, réseaux, systèmes, programmation.

Tu apprendras d'abord à compter comme une machine : convertir les nombres entre les bases 10, 2, 8 et 16, puis additionner, soustraire et multiplier directement en binaire, sans repasser par le décimal. Viendra ensuite la question des nombres négatifs, résolue par le complément à deux, le codage que tous les processeurs actuels utilisent pour les entiers signés. Enfin, nous appliquerons la même logique de codage aux autres types d'information : les caractères avec ASCII puis Unicode, les images et les sons avec leurs calculs de taille, et même la fiabilité des transmissions avec le bit de parité, qui permet de repérer certaines erreurs.

Aucun prérequis n'est nécessaire au-delà des quatre opérations et des puissances de 2, rappelées au fur et à mesure. La méthode de travail compte autant que les résultats : chaque conversion et chaque opération doit être vérifiée, soit en effectuant la conversion inverse, soit en contrôlant le résultat en décimal, soit en le recalculant par un petit script, comme le propose le TP de fin de cours. En numération, une erreur d'un seul bit fausse tout le résultat : l'habitude de vérifier, acquise ici, te suivra partout.

Prends donc un papier et un crayon : ce cours se travaille la main dans les calculs, pas seulement les yeux sur les pages.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Convertir un nombre entre les bases 2, 8, 10 et 16, par les divisions successives, la somme pondérée et les regroupements de bits.
- Effectuer additions, soustractions et multiplications directement en binaire, en gérant retenues et emprunts.
- Coder un entier négatif en complément à deux sur 8 bits et additionner des nombres signés.
- Donner la plage de valeurs représentables sur n bits, en non signé et en complément à deux.
- Coder un caractère et un court texte en ASCII, et expliquer ce qu'Unicode et UTF-8 ajoutent pour les accents et les symboles du monde entier.
- Calculer la taille brute d'une image (résolution × profondeur de couleur) et d'un son (fréquence × résolution × canaux × durée).
- Ajouter un bit de parité à un mot binaire, contrôler un mot reçu et énoncer les limites de cette détection d'erreurs.

Leçon 1 — Base 10, base 2, base 8 et base 16 : les conversions

Un système de numération est défini par sa base, c'est-à-dire le nombre de symboles (ou chiffres) qu'il utilise. La base 10 emploie les chiffres de 0 à 9 : c'est notre système quotidien, hérité de nos dix doigts. La base 2, dite binaire, n'a que deux chiffres, 0 et 1 : elle épouse parfaitement les circuits électroniques, qui distinguent deux états. La base 8 (octal) utilise les chiffres de 0 à 7, et la base 16 (hexadécimal) les chiffres de 0 à 9 complétés par les lettres A, B, C, D, E, F, qui valent 10, 11, 12, 13, 14 et 15. Dans tous les systèmes, la valeur d'un chiffre dépend de sa position : le chiffre situé au rang k (en comptant à partir de 0, depuis la droite) pèse base puissance k. Ainsi, 45 en base 10 vaut 4 × 10 + 5, et le même principe s'applique à toutes les bases.

Pour convertir un nombre vers le décimal, on applique cette définition : on multiplie chaque chiffre par la puissance de la base correspondant à son rang, et on additionne. C'est la méthode de la somme pondérée. Pour convertir depuis le décimal vers une base b, on procède par divisions successives : on divise le nombre par b, on note le reste, on recommence avec le quotient, jusqu'à obtenir un quotient nul ; le résultat se lit en remontant les restes, du dernier au premier. Exemple complet dans l'encadré ci-dessous : 45 devient 101101 en binaire, et la somme pondérée redonne bien 32 + 8 + 4 + 1 = 45. Cette vérification par le chemin inverse doit devenir un réflexe systématique.

Les bases 2, 8 et 16 sont des puissances de 2, ce qui offre un raccourci précieux : chaque chiffre octal correspond exactement à un groupe de 3 bits, et chaque chiffre hexadécimal à un groupe de 4 bits. Pour passer du binaire à l'octal ou à l'hexadécimal,

on regroupe donc les bits par paquets de 3 ou de 4 en partant de la droite (en complétant par des zéros à gauche si besoin), puis on traduit chaque paquet séparément. Dans l'autre sens, on éclate chaque chiffre en son groupe de bits. C'est pour cette lisibilité que l'hexadécimal sert d'écriture compacte du binaire : adresses mémoire, couleurs d'écran (trois octets, un par composante rouge, verte et bleue) ou codes d'erreur s'écrivent en hexadécimal plutôt qu'en longues suites de 0 et de 1.

L'essentiel
Une base b utilise b symboles, et chaque position vaut b élevé au rang. Vers le décimal : somme pondérée. Depuis le décimal : divisions successives, restes lus à l'envers. Entre binaire, octal et hexadécimal : regroupements de 3 ou 4 bits.

Exemple — Convertir 45 dans les bases 2, 8 et 16 — et vérifier

```
Convertir 45 (base 10) en binaire : divisions par 2
45 / 2 = 22, reste 1
22 / 2 = 11, reste 0
11 / 2 = 5, reste 1
5 / 2 = 2, reste 1
2 / 2 = 1, reste 0
1 / 2 = 0, reste 1
On lit les restes de bas en haut : 45 = 101101 (base 2)
Verification : 32 + 8 + 4 + 1 = 45. Correct.

Regroupements, sans repasser par le decimal :
101101 -> groupes de 3 bits : 101 | 101 -> 5 et 5
donc 45 = 55 (base 8)
101101 -> groupes de 4 bits : 0010 | 1101 -> 2 et D
donc 45 = 2D (base 16)

Convertir 3F (base 16) en decimal :
3F = 3 x 16 + 15 = 48 + 15 = 63
et en binaire, chiffre par chiffre : 3 = 0011, F = 1111
donc 3F = 0011 1111 (base 2), et 32+16+8+4+2+1 = 63.
```

À retenir

- Le poids d'un chiffre est la base élevée à son rang ; le rang se compte à partir de 0 en partant de la droite.
- Décimal vers base b : divisions successives par b ; les restes se lisent de bas en haut, le dernier reste est le chiffre de gauche.
- Binaire ↔ octal par paquets de 3 bits, binaire ↔ hexadécimal par paquets de 4 bits, toujours en regroupant depuis la droite.

Exercice

1) Convertis 27 (base 10) en binaire par divisions successives, puis en hexadécimal par regroupement. 2) Convertis 100011 (base 2) en décimal par la somme pondérée. 3) Convertis 2A (base 16) en décimal, puis en binaire. 4) Convertis 56 (base 8) en décimal.

✔ Corrigé
1) Divisions : 27 / 2 = 13 reste 1 ; 13 / 2 = 6 reste 1 ; 6 / 2 = 3 reste 0 ; 3 / 2 = 1 reste 1 ; 1 / 2 = 0 reste 1. Restes lus de bas en haut : 11011. Vérification : 16 + 8 + 2 + 1 = 27. Regroupement par 4 bits : 0001 | 1011 → 1 et B, donc 27 = 1B en base 16 (contrôle : 1 × 16 + 11 = 27). 2) 100011 = $1\times2^5 + 0\times2^4 + 0\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$ = 32 + 2 + 1 = 35. 3) 2A = 2 × 16 + 10 = 42 ; en binaire : 2 = 0010 et A = 1010, donc 0010 1010 (contrôle : 32 + 8 + 2 = 42). 4) 56 en base 8 = 5 × 8 + 6 = 46.

Leçon 2 — Opérations arithmétiques en binaire

L'addition binaire suit exactement les mêmes règles qu'en décimal, colonne par colonne, de droite à gauche — mais la retenue arrive beaucoup plus tôt, dès que la somme d'une colonne atteint 2. La table complète tient en quatre lignes : 0 + 0 = 0 ; 0 + 1 = 1 ; 1 + 1 = 0 et je retiens 1 ; 1 + 1 + 1 (avec la retenue entrante) = 1 et je retiens 1. Dans l'exemple ci-dessous, 1011 + 0110 (c'est-à-dire 11 + 6) produit des retenues sur trois colonnes et donne 10001, soit 17 : on remarque que le résultat peut avoir un bit de plus que les deux opérandes, c'est la retenue finale qui déborde à gauche.

La soustraction fonctionne par emprunt, comme à l'école primaire, mais on emprunte 2 au lieu de 10 : si le chiffre du haut est plus petit que celui du bas, on ajoute 2 au chiffre du haut et on diminue de 1 le chiffre de la colonne suivante. L'exemple détaillé montre 1101 − 0111 (13 − 7) avec ses deux emprunts en cascade. À titre professionnel, retiens dès maintenant qu'il existe une autre méthode, bien plus élégante : remplacer la soustraction par l'addition de l'opposé codé en complément à deux. C'est elle que les processeurs utilisent réellement, car un additionneur suffit alors pour les deux opérations — nous la construisons à la leçon suivante.

La multiplication binaire ressemble à la multiplication posée décimale, en beaucoup plus simple : chaque chiffre du multiplicateur ne vaut que 0 ou 1, donc chaque produit partiel est soit une ligne de zéros, soit la copie du multiplicande décalée d'un rang vers la gauche. On additionne ensuite toutes les lignes. Avec 101 × 11 (5 × 3), il n'y a que deux lignes à additionner, et le résultat 1111 se contrôle en décimal : 8 + 4 + 2 + 1 = 15 = 5 × 3. Cette habitude du contrôle décimal — convertir les opérandes et le résultat, comparer — reste la manière la plus sûre de valider n'importe quel calcul binaire.

L'essentiel
En binaire, la retenue vaut 2 : 1 + 1 = 0, retenue 1. La soustraction emprunte 2 à la colonne voisine — ou, mieux, se ramène à une addition en complément à deux. La multiplication n'utilise que des produits partiels décalés, valant 0 ou le multiplicande.

Exemple — Addition, soustraction et multiplication posées en binaire

```
Addition posee : 1011 + 0110 (c'est 11 + 6)
retenues : 1 1 1
1 0 1 1
+ 0 1 1 0
-----------
= 1 0 0 0 1 (16 + 1 = 17, et 11 + 6 = 17)

Soustraction posee : 1101 - 0111 (c'est 13 - 7)
bit 0 : 1 - 1 = 0
bit 1 : 0 - 1 -> on emprunte : 2 - 1 = 1 (le bit 2 diminue)
bit 2 : 0 - 1 -> on emprunte : 2 - 1 = 1 (le bit 3 diminue)
bit 3 : 0 - 0 = 0
Resultat : 0110 (4 + 2 = 6, et 13 - 7 = 6)

Multiplication posee : 101 x 11 (c'est 5 x 3)
1 0 1
x 1 1
---------
1 0 1 (101 x 1)
1 0 1 0 (101 x 1, decale d'un rang)
---------
= 1 1 1 1 (8 + 4 + 2 + 1 = 15, et 5 x 3 = 15)
```

À retenir

• 1 + 1 = 0, retenue 1 : toute l'arithmétique binaire repose sur cette ligne ; une retenue peut se propager sur plusieurs colonnes.

• Emprunter dans une soustraction ajoute 2 au chiffre (et non 10 comme en décimal) et diminue le chiffre voisin de 1.

• Toujours vérifier une opération binaire en reconvertissant opérandes et résultat en décimal : les deux calculs doivent raconter la même histoire.

Exercice

1) Pose et calcule 1100 (base 2) + 1010 (base 2), puis vérifie en décimal. 2) Pose et calcule 10101 (base 2) − 01011 (base 2) en détaillant les emprunts, puis vérifie. 3) Calcule 110 (base 2) × 10 (base 2) par produits partiels décalés.

✔ Corrigé

1) 1100 + 1010 : bit 0 : 0 + 0 = 0 ; bit 1 : 0 + 1 = 1 ; bit 2 : 1 + 0 = 1 ; bit 3 : 1 + 1 = 0, retenue 1 ; bit 4 : la retenue donne 1. Résultat : 10110. Vérification : 1100 = 12, 1010 = 10, 12 + 10 = 22, et 10110 = 16 + 4 + 2 = 22. 2) 10101 − 01011 : bit 0 : 1 − 1 = 0 ; bit 1 : 0 − 1 → emprunt : 2 − 1 = 1, le bit 2 passe de 1 à 0 ; bit 2 : 0 − 0 = 0 ; bit 3 : 0 − 1 → emprunt : 2 − 1 = 1, le bit 4 passe de 1 à 0 ; bit 4 : 0 − 0 = 0. Résultat : 01010. Vérification : 10101 = 21, 01011 = 11, 21 − 11 = 10, et 01010 = 8 + 2 = 10. 3) 110 × 10 : le chiffre 0 du multiplicateur donne une ligne nulle, le chiffre 1 donne le multiplicande décalé d'un rang : 1100. Résultat 1100 = 12, et 6 × 2 = 12.

Leçon 3 — Représentation des nombres négatifs : le complément à deux

Un ordinateur doit aussi manipuler des nombres négatifs : températures sous zéro, soldes débiteurs, altitudes relatives. Première idée, le bit de signe : réserver le bit de gauche pour dire + ou −, et coder la valeur absolue avec les autres bits. Elle échoue deux fois : zéro y possède deux écritures (+0 et −0), et surtout l'addition ordinaire donne des résultats faux dès qu'un négatif intervient, ce qui obligerait le processeur à des circuits spéciaux. Le complément à un (inverser tous les bits pour changer de signe) garde le même défaut du double zéro. Il fallait une représentation où l'addition binaire habituelle fonctionne pour tous les nombres, positifs comme négatifs, sans aucun circuit supplémentaire.

Cette représentation est le complément à deux, utilisé par tous les processeurs actuels. Pour coder −x sur n bits : on écrit x en binaire sur n bits (avec des zéros à gauche), on inverse tous les bits — c'est le complément à un — puis on ajoute 1. Le bit de gauche joue alors automatiquement le rôle de bit de signe : 0 pour les positifs, 1 pour les négatifs. Pour relire un mot dont le bit de gauche vaut 1, on peut appliquer la même opération (inverser, ajouter 1) pour retrouver la valeur absolue, ou utiliser la formule : valeur = valeur non signée du mot − 2^n. Ainsi 11111011, qui vaut 251 en non signé, représente 251 − 256 = −5 sur 8 bits, comme le montre l'exemple.

La plage représentable mérite d'être connue par cœur. Sur n bits non signés : de 0 à 2^n − 1, soit de 0 à 255 sur 8 bits. En complément à deux : de -2^{n-1} à 2^{n-1} − 1, soit de −128 à +127 sur 8 bits. La plage est asymétrique parce qu'il n'y a qu'un seul zéro, rangé du côté des positifs : il reste donc une valeur négative de plus. Conséquence pratique, le dépassement de capacité (overflow) : si le vrai résultat d'un calcul sort de la plage, par exemple 100 + 50 = 150 sur 8 bits signés, le mot obtenu est faux sans qu'aucune retenue ne le signale de façon évidente. Repérer ce cas — la somme de deux nombres de même signe qui change de signe — fait partie des réflexes à acquérir.

L'essentiel
En complément à deux, −x s'obtient en inversant les bits de x puis en ajoutant 1. Sur 8 bits, la plage signée va de −128 à +127 ; le bit de gauche indique le signe, et l'addition binaire ordinaire fonctionne pour tous les nombres.

Exemple — Coder −5 sur 8 bits, le vérifier, puis additionner 25 + (−9)

```
Coder -5 sur 8 bits :
+5 : 0000 0101
bits inverses : 1111 1010 (complement a un)
+ 1 : 1111 1011 <- c'est -5
Verification 1 : 1111 1011 lu en non signe vaut 251,
et 251 - 256 = -5. Correct.
Verification 2 : (+5) + (-5) doit donner 0 :
0000 0101 + 1111 1011 = 1 0000 0000
la retenue finale sort des 8 bits, il reste 0. Correct.

Addition signee : 25 + (-9)
25 = 0001 1001
-9 : 9 = 0000 1001 -> inverse 1111 0110 -> +1 : 1111 0111
somme : 0001 1001 + 1111 0111 = 1 0001 0000
on garde 8 bits : 0001 0000 = 16, et 25 - 9 = 16. Correct.
```

À retenir

• Passer de x à −x, ou de −x à x : toujours la même recette — inverser tous les bits et ajouter 1.

• Sur n bits en complément à deux : minimum = -2^{n-1}, maximum = $2^{n-1} - 1$; sur 8 bits, de −128 à +127.

• Dépassement : si l'addition de deux nombres de même signe produit un résultat de signe opposé, le résultat est faux (exemple : 100 + 50 sur 8 bits signés).

Exercice

1) Code −12 sur 8 bits en complément à deux, en détaillant les étapes, et vérifie avec la formule « valeur non signée − 256 ». 2) Effectue en binaire signé l'opération 30 + (−12) et vérifie le résultat. 3) Peut-on coder +200 et −200 sur 8 bits en complément à deux ? Justifie, et donne le nombre minimal de bits nécessaire.

✔ Corrigé
1) 12 = 0000 1100 ; inversion des bits : 1111 0011 ; ajout de 1 : 1111 0100. Vérification : 1111 0100 vaut 244 en non signé, et 244 − 256 = −12. 2) 30 = 0001 1110 et −12 = 1111 0100. Addition : 0001 1110 + 1111 0100 = 1 0001 0010 ; on garde les 8 bits de droite : 0001 0010 = 16 + 2 = 18. Vérification : 30 − 12 = 18. 3) Non : la plage signée sur 8 bits s'arrête à +127 et −128, donc 200 comme −200 sont hors plage. Il faut au moins 9 bits, dont la plage signée va de −256 à +255, ce qui contient bien −200 et +200.

Leçon 4 — Codage des caractères : ASCII et Unicode

Un caractère n'est pas un nombre : impossible d'additionner des lettres. Pour coder du texte, on convient donc d'une table qui associe à chaque caractère un numéro, et c'est ce numéro que l'on code en binaire. La table historique est ASCII (American Standard Code for Information Interchange), créée aux États-Unis dans les années 1960. Elle utilise 7 bits, donc 128 numéros, de 0 à 127 : les lettres latines majuscules et minuscules, les chiffres, la ponctuation, l'espace, et des caractères dits de contrôle,

comme le retour à la ligne (numéro 10), invisibles mais indispensables. En mémoire, chaque caractère ASCII occupe presque toujours un octet complet, dont le premier bit reste à 0.

Quelques repères de la table rendent les calculs rapides. « A » vaut 65, et les majuscules se suivent dans l'ordre alphabétique : « B » = 66, « C » = 67, jusqu'à « Z » = 90. Les minuscules commencent à « a » = 97 : l'écart entre une minuscule et sa majuscule vaut exactement 32, ce qui correspond à un seul bit de différence — un clin d'œil des concepteurs qui simplifie les conversions de casse. Attention au piège classique : le caractère « 0 » vaut 48, ce n'est pas le nombre zéro. Coder un mot revient à juxtaposer les octets de ses lettres : « IAM » devient la suite des codes 73, 65, 77, soit trois octets en binaire, comme le montre l'exemple.

ASCII a une limite évidente : pensée pour l'anglais, la table ne contient ni les lettres accentuées (é, è, ç, à), ni les autres alphabets du monde, ni les idéogrammes, ni les émojis. Des extensions régionales sur 8 bits ont dépanné un temps, mais en se contredisant d'un pays à l'autre. La solution universelle est Unicode : un répertoire qui attribue à chaque caractère de chaque écriture un identifiant unique, son point de code, noté U+ suivi d'un nombre hexadécimal. « A » reste U+0041 (donc 65, compatibilité oblige), « é » est U+00E9, et un émoji comme le visage souriant est U+1F600. Pour stocker ces points de code, l'encodage UTF-8, devenu le standard du Web, utilise 1 octet pour les caractères ASCII, 2 octets pour la plupart des caractères accentués, et jusqu'à 4 octets pour les émojis : la taille d'un texte dépend donc des caractères qu'il contient.

L'essentiel
Coder un texte = associer à chaque caractère un numéro (une table), puis coder ce numéro en binaire. ASCII : 128 caractères sur 7 bits, « A » = 65. Unicode attribue un point de code U+XXXX à tous les caractères du monde ; UTF-8 le stocke en 1 à 4 octets.

Exemple — Coder « IAM » en ASCII, et situer ASCII, Unicode, UTF-8

```
Coder le mot IAM en ASCII (1 octet par caractere) :
I = 73 = 0100 1001
A = 65 = 0100 0001
M = 77 = 0100 1101
IAM -> 01001001 01000001 01001101 (3 octets)

Reperes de la table ASCII :
espace = 32 '0' = 48 'A' = 65 'a' = 97
'B' = 66 : les majuscules se suivent par ordre alphabetique
'a' - 'A' = 32 : passer en minuscule = ajouter 32

Avec Unicode et son encodage UTF-8 :
'A' = U+0041 -> 1 octet en UTF-8 : 41 (en hexadecimal)
'e accent aigu' = U+00E9 -> 2 octets : C3 A9
visage souriant = U+1F600 -> 4 octets : F0 9F 98 80
```

À retenir

• Le caractère « 0 » (code 48) n'est pas le nombre 0 : un chiffre dans un texte est un caractère comme un autre, avec son propre code.

• ASCII ne connaît ni accents ni alphabets non latins : les erreurs d'encodage (caractères affichés de travers) viennent presque toujours d'une mauvaise lecture de la table ou de l'encodage.

• UTF-8 utilise une taille variable : 1 octet pour l'ASCII historique, 2 pour la plupart des lettres accentuées, jusqu'à 4 pour les émojis — d'où des tailles de fichiers texte qui varient selon le contenu.

Exercice

1) Code le mot « OK » en ASCII : donne les codes décimaux de chaque lettre, puis les deux octets en binaire. 2) Un texte de 2 000 caractères ne contient que des caractères ASCII : quelle est sa taille en mémoire ? 3) Le même texte est réécrit en UTF-8, avec 1 900 caractères ASCII et 100 lettres accentuées comme « é » : quelle est sa nouvelle taille ? Explique la différence.

✔ Corrigé

1) « O » = 79 et « K » = 75 (les majuscules se suivent à partir de A = 65 : O est la 15e lettre, 65 + 14 = 79 ; K est la 11e, 65 + 10 = 75). En binaire : 79 = 64 + 8 + 4 + 2 + 1 = 0100 1111, et 75 = 64 + 8 + 2 + 1 = 0100 1011. Le mot « OK » occupe donc les deux octets 01001111 01001011. 2) En ASCII, un caractère = un octet : 2 000 caractères = 2 000 octets. 3) En UTF-8 : les 1 900 caractères ASCII occupent 1 octet chacun, soit 1 900 octets ; les 100 lettres accentuées occupent 2 octets chacune, soit 200 octets ; total 2 100 octets. Le texte « gonfle » de 100 octets parce que chaque lettre accentuée coûte un octet de plus en UTF-8 qu'un caractère ASCII.

Leçon 5 — Codage des images et du son : initiation

Une image numérique est une grille de points élémentaires, les pixels, rangés en lignes et en colonnes : sa résolution s'exprime en largeur × hauteur, par exemple 800 × 600, soit 480 000 pixels. Chaque pixel reçoit une couleur codée en binaire, et le nombre de bits consacrés à un pixel s'appelle la profondeur de couleur. Avec 1 bit par pixel, on ne distingue que deux couleurs (noir et blanc). Avec 8 bits, on obtient 256 valeurs : assez pour des nuances de gris ou une palette de 256 couleurs. Avec 24 bits — 3 octets, un pour chaque composante rouge, verte et bleue, codée de 0 à 255 — on atteint 2^{24} = 16 777 216 couleurs, plus que l'œil ne peut en distinguer. La taille brute d'une image se calcule donc simplement : nombre de pixels × nombre d'octets par pixel.

Ce calcul explique des ordres de grandeur que tout informaticien doit sentir. Une image 800 × 600 en 24 bits pèse 480 000 × 3 = 1 440 000 octets, environ 1,44 Mo. Une image Full HD de 1920 × 1080 pixels grimpe à 6 220 800 octets, environ 6,22 Mo, et la photo d'un smartphone de 4000 × 3000 pixels atteindrait 36 Mo sans traitement. Retiens aussi que doubler à la fois la largeur et la hauteur multiplie la taille par quatre, et non par deux. En pratique, les formats JPEG ou PNG compressent ces données (JPEG avec une légère perte de qualité, PNG sans perte) : les fichiers réels sont donc

nettement plus petits que la taille brute, qui reste la référence pour les calculs et pour comprendre pourquoi la compression est indispensable.

Le son suit la même logique, mais en une dimension : le temps. Échantillonner un son consiste à mesurer l'amplitude du signal à intervalles réguliers. Trois paramètres décrivent le codage : la fréquence d'échantillonnage, en hertz (nombre de mesures par seconde) ; la résolution, en bits par échantillon (la précision de chaque mesure) ; et le nombre de canaux (1 en mono, 2 en stéréo). La taille brute vaut alors : fréquence × (résolution ÷ 8) × canaux × durée en secondes. Le CD audio utilise 44 100 Hz en 16 bits stéréo, soit 44 100 × 2 × 2 = 176 400 octets par seconde : une chanson de trois minutes occuperait 176 400 × 180 = 31 752 000 octets, environ 31,8 Mo, sans compression. Le format MP3, qui supprime des sons peu perceptibles, divise cette taille par environ dix — au prix d'une perte, là encore.

L'essentiel
Taille brute d'une image = nombre de pixels × octets par pixel. Taille brute d'un son = fréquence d'échantillonnage × octets par échantillon × canaux × durée. JPEG, PNG et MP3 compressent ces données brutes, avec ou sans perte.

Exemple — Calculer la taille brute d'images et d'un son, pas à pas

```
Image 800 x 600 pixels, couleurs 24 bits (3 octets/pixel) :
pixels = 800 x 600 = 480 000
taille = 480 000 x 3 = 1 440 000 octets (environ 1,44 Mo)
Image Full HD 1920 x 1080, 24 bits :
2 073 600 pixels x 3 = 6 220 800 octets (environ 6,22 Mo)
Image noir et blanc 1024 x 768, 1 bit par pixel :
786 432 bits / 8 = 98 304 octets (environ 98 Ko)

Son qualite CD : 44 100 Hz, 16 bits (2 octets), stereo (2 canaux)
par seconde : 44 100 x 2 x 2 = 176 400 octets
pour 3 minutes (180 s) : 176 400 x 180 = 31 752 000 octets
soit environ 31,8 Mo, avant compression.
```

À retenir

- 24 bits par pixel = 3 octets, un par composante rouge, verte, bleue, chacune codée sur 256 niveaux (0 à 255).
- Doubler la largeur ET la hauteur d'une image multiplie sa taille brute par 4 : la résolution croît comme une surface.
- Pour le son, retenir la formule complète avec les canaux : oublier la stéréo (× 2) est l'erreur la plus fréquente dans ces calculs.

Exercice

1) Calcule la taille brute d'une image de 640 × 480 pixels codée en 256 couleurs (8 bits par pixel). 2) Calcule la taille brute d'un message vocal de 10 secondes échantillonné à 8 000 Hz, en 8 bits, en mono. 3) Une chanson MP3 de trois minutes pèse environ 3 Mo, alors que le calcul du cours donne 31,8 Mo en qualité CD : explique

cet écart.

✔ Corrigé
1) Nombre de pixels : 640 × 480 = 307 200. En 256 couleurs, chaque pixel tient sur 8 bits, c'est-à-dire 1 octet. Taille brute = 307 200 × 1 = 307 200 octets, environ 307 Ko. 2) Par seconde, le codage produit 8 000 échantillons × 1 octet × 1 canal = 8 000 octets. Pour 10 secondes : 8 000 × 10 = 80 000 octets, soit 80 Ko — voilà pourquoi la voix téléphonique, codée ainsi, occupe si peu de place. 3) Les 31,8 Mo correspondent au signal brut, sans aucune compression (format WAV ou piste CD). Le MP3 applique une compression avec perte d'environ 10 pour 1 : il élimine notamment des fréquences que l'oreille perçoit à peine. 31,8 ÷ 10 ≈ 3,2 Mo : on retombe sur la taille observée d'environ 3 Mo.

Leçon 6 — Codes détecteurs d'erreurs : le bit de parité

Coder l'information ne suffit pas : encore faut-il qu'elle survive au voyage. Or les transmissions réelles sont imparfaites — parasites électriques, interférences radio, rayures sur un support : un bit peut basculer de 0 à 1 ou de 1 à 0 pendant le transfert, et le récepteur reçoit alors un mot différent de celui qui a été émis, sans aucun moyen de s'en apercevoir. Pour se protéger, on ajoute à l'information utile une information de contrôle, calculée à partir des données selon une règle convenue : c'est le principe de tous les codes détecteurs d'erreurs. La règle la plus simple qui soit utilise un unique bit supplémentaire, le bit de parité.

Le principe est le suivant. En parité paire, l'émetteur compte le nombre de bits à 1 dans le mot de données : si ce nombre est déjà pair, le bit de parité ajouté vaut 0 ; s'il est impair, il vaut 1. Dans les deux cas, le mot transmis contient un nombre pair de bits à 1. (La convention inverse, la parité impaire, force un total impair.) À la réception, on recompte : si le total est devenu impair alors qu'on attendait un nombre pair, la parité est rompue — une erreur est détectée, et le récepteur peut rejeter le mot ou demander sa retransmission. L'exemple ci-dessous applique la méthode à deux mots de 7 bits, puis simule un mot reçu défectueux.

Il faut être lucide sur les limites de ce code minimal. Il détecte toute erreur touchant un nombre impair de bits (une, trois, cinq erreurs…), car chacune bascule la parité. Mais deux erreurs simultanées se compensent : le total redevient pair et le mot faux est accepté comme correct. De plus, même lorsqu'il détecte, le bit de parité ne dit pas où se trouve l'erreur : il détecte sans corriger, et seule une retransmission peut réparer. C'est pourquoi les systèmes réels emploient des codes plus riches — sommes de contrôle, codes CRC utilisés dans les réseaux, codes de Hamming qui détectent et localisent l'erreur pour la corriger — mais tous prolongent la même idée fondatrice rencontrée ici : ajouter une redondance calculée pour rendre les erreurs visibles.

L'essentiel
Le bit de parité ajoute un bit calculé pour que le nombre total de bits à 1 du mot transmis ait une parité fixée (paire ou impaire). Il détecte toute erreur touchant un nombre impair de bits, mais il ne corrige rien et laisse passer les erreurs doubles.

Exemple — Parité paire sur des mots de 7 bits : calcul et contrôle

```
Parite paire : on ajoute un bit pour que le total de 1 soit pair.

Donnees 1011011 : le bit 1 apparait 5 fois (nombre impair)
-> bit de parite = 1 pour rendre le total pair
mot transmis : 1011011 1 (6 fois le bit 1 : pair, correct)

Donnees 1011001 : le bit 1 apparait 4 fois (nombre pair)
-> bit de parite = 0
mot transmis : 1011001 0

Controle a la reception : on recoit le mot 10110011
nombre de bits a 1 : 5, un nombre impair
alors qu'un total pair etait attendu
-> ERREUR DETECTEE : le mot est rejete ou redemande.
```

À retenir

- En parité paire, le bit ajouté vaut 0 si le nombre de 1 des données est déjà pair, et 1 sinon ; en parité impaire, c'est l'inverse.
- Une erreur unique est toujours détectée ; deux erreurs simultanées se compensent et passent inaperçues : toujours énoncer cette limite.
- Détecter n'est pas corriger : le bit de parité ne localise pas l'erreur, le récepteur ne peut que rejeter le mot ou demander une retransmission.

Exercice

1) Calcule le bit de parité paire à ajouter au mot de données 0110101, puis écris le mot complet transmis. 2) En réception (parité paire), on lit le mot 01101011 : est-il accepté ou rejeté ? Justifie par le comptage. 3) Un mot correct transmis en parité paire était 10110111. Pendant le trajet, deux bits changent et l'on reçoit 10010011. Le contrôle de parité détecte-t-il l'erreur ? Explique ce que cela révèle sur le code.

✔ Corrigé
1) Le mot 0110101 contient le bit 1 quatre fois : 0, puis 1, 1, 0, 1, 0, 1 — soit 4, un nombre pair. En parité paire, le bit à ajouter vaut donc 0, et le mot transmis est 01101010, qui contient bien un nombre pair de 1. 2) Le mot reçu 01101011 contient le bit 1 cinq fois : c'est un nombre impair, alors que la convention exige un total pair. La parité est rompue : le mot est rejeté, une erreur s'est produite pendant la transmission. 3) Le mot reçu 10010011 contient le bit 1 quatre fois : le total est pair, donc le contrôle accepte le mot. Pourtant il est faux — deux bits ont changé par rapport au mot émis (qui contenait 6 fois le bit 1). Les deux erreurs se sont compensées du point de vue de la parité : c'est la limite fondamentale du bit de parité, qui ne détecte que les erreurs en nombre impair.

Travaux dirigés (TD)

TD 1 — Un même nombre dans toutes les bases

On considère le nombre décimal 156. 1) Convertis-le en binaire par divisions successives. 2) À partir de l'écriture binaire, obtiens son écriture en octal puis en hexadécimal par regroupements de bits. 3) Convertis le nombre hexadécimal C5 en décimal, puis en binaire. 4) Effectue directement en binaire l'addition 10011100 + 00101101 (pose les retenues), et vérifie le résultat en décimal en remarquant que ces deux nombres sont 156 et 45.

✔ Corrigé du TD 1

1) Divisions par 2 : 156 = 2 × 78 + 0 ; 78 = 2 × 39 + 0 ; 39 = 2 × 19 + 1 ; 19 = 2 × 9 + 1 ; 9 = 2 × 4 + 1 ; 4 = 2 × 2 + 0 ; 2 = 2 × 1 + 0 ; 1 = 2 × 0 + 1. Les restes lus de bas en haut donnent 10011100. Vérification : 128 + 16 + 8 + 4 = 156. 2) Octal, groupes de 3 bits : 010 | 011 | 100 → 2, 3, 4, donc 234 en base 8 (contrôle : 2 × 64 + 3 × 8 + 4 = 128 + 24 + 4 = 156). Hexadécimal, groupes de 4 bits : 1001 | 1100 → 9 et C, donc 9C en base 16 (contrôle : 9 × 16 + 12 = 156). 3) C5 = 12 × 16 + 5 = 192 + 5 = 197. En binaire : C = 1100 et 5 = 0101, donc 11000101 (contrôle : 128 + 64 + 4 + 1 = 197). 4) Addition posée : bit 0 : 0 + 1 = 1 ; bit 1 : 0 + 0 = 0 ; bit 2 : 1 + 1 = 0, retenue 1 ; bit 3 : 1 + 1 + 1 = 1, retenue 1 ; bit 4 : 0 + 0 + 1 = 1 ; bit 5 : 0 + 1 = 1 ; bit 6 : 0 + 0 = 0 ; bit 7 : 1 + 0 = 1. Résultat : 11001001. Vérification : 156 + 45 = 201, et 11001001 = 128 + 64 + 8 + 1 = 201.

TD 2 — Signés, parité et taille d'une photo

1) Code le nombre −64 sur 8 bits en complément à deux, en détaillant les étapes, puis vérifie avec la formule « valeur non signée − 256 ». 2) En parité impaire cette fois, quel bit de parité faut-il ajouter au mot de données 0101110 ? Écris le mot transmis et justifie le comptage. 3) Une photo de smartphone a une résolution de 4000 × 3000 pixels, codée en 24 bits par pixel. Calcule sa taille brute en octets puis en mégaoctets. Combien de photos brutes de ce type tiendraient sur un espace de 16 000 Mo ? Pourquoi en stocke-t-on en réalité bien davantage ?

✔ Corrigé du TD 2

1) 64 en binaire sur 8 bits : 0100 0000. Inversion des bits : 1011 1111. Ajout de 1 : 1100 0000. C'est le codage de −64. Vérification : 1100 0000 vaut 192 en non signé, et 192 − 256 = −64. 2) Le mot 0101110 contient le bit 1 quatre fois (0, 1, 0, 1, 1, 1, 0). En parité impaire, le total de bits à 1 du mot transmis doit être impair : il faut donc ajouter un bit de parité égal à 1. Mot transmis : 01011101, qui contient 5 fois le bit 1, un total impair comme exigé. 3) Nombre de pixels : 4000 × 3000 = 12 000 000. À 3 octets par pixel : 12 000 000 × 3 = 36 000 000 octets, soit 36 Mo. Nombre de photos brutes : 16 000 ÷ 36 ≈ 444,4, donc 444 photos complètes. En réalité, les smartphones enregistrent en JPEG, un format compressé (souvent dix fois plus léger, au prix d'une petite perte) : on stocke donc plusieurs milliers de photos dans le même espace.

Travaux pratiques (TP)

TP 1 — Vérifier tout le cours avec un petit script Python

L'objectif de ce TP est de confronter tes calculs à la main avec ceux de la machine. 1) Écris un script qui demande un entier décimal et affiche ses écritures binaire, octale et hexadécimale (fonctions bin, oct, hex, ou format). AVANT de lancer le script, convertis 45 et 156 à la main sur papier ; compare ensuite. 2) Programme une fonction complement_a_deux(n) qui renvoie la chaîne des 8 bits de n codé en complément à deux (astuce : calculer n modulo 256, puis formater sur 8 bits). Teste-la pour −5, −12 et −64 et compare avec les résultats du cours. 3) Programme le calcul du bit de parité paire d'un mot de 7 bits donné comme chaîne de caractères ; ajoute le bit, simule une erreur en inversant un bit du mot transmis et vérifie que ton contrôle détecte l'erreur ; recommence avec deux bits inversés et observe la limite du code. 4) Fais calculer par le script les tailles de la leçon 5 : image 1920 × 1080 en 24 bits, et son CD de 3 minutes. Toute différence avec tes calculs manuels doit être expliquée dans le compte rendu.

✔ Attendus du TP 1

Attendus et valeurs de contrôle. 1) Pour 45 : binaire 101101, octal 55, hexadécimal 2D. Pour 156 : binaire 10011100, octal 234, hexadécimal 9C. Si une seule de ces valeurs diffère de ton calcul manuel, refais ce calcul précis : l'écart localise l'erreur. 2) complement_a_deux(−5) renvoie 11111011 ; (−12) → 11110100 ; (−64) → 11000000 : ce sont exactement les codages construits dans les leçons et exercices. Une implémentation correcte repose sur le fait que le complément à deux de n sur 8 bits est la représentation binaire de n modulo 256. 3) Pour le mot 1011011 (cinq bits à 1), le bit de parité paire vaut 1 et le mot transmis est 10110111. Après inversion d'un seul bit, le nombre de 1 devient impair : le contrôle signale l'erreur. Après inversion de deux bits, le nombre de 1 redevient pair : le contrôle ne voit rien — le script doit reproduire exactement cette limite théorique. 4) Image Full HD : 1920 × 1080 × 3 = 6 220 800 octets. Son CD : 44 100 × 2 × 2 × 180 = 31 752 000 octets. Compte rendu attendu : les sorties du script recopiées, la comparaison ligne à ligne avec les calculs manuels, et une phrase sur chaque écart éventuel.

Exercices corrigés supplémentaires

Exercice 1

Convertis le nombre binaire 11101011 en hexadécimal (par regroupement), puis en décimal de deux manières : par la somme pondérée et à partir de l'écriture hexadécimale.

✔ Corrigé

Regroupement par 4 bits : 1110 | 1011 → E et B, donc 11101011 = EB en base 16. Somme pondérée : 128 + 64 + 32 + 0 + 8 + 0 + 2 + 1 = 235. À partir de l'hexadécimal : E × 16 + B = 14 × 16 + 11 = 224 + 11 = 235. Les deux chemins donnent le même résultat, 235 : la conversion est cohérente.

Exercice 2

Sur 8 bits en complément à deux, effectue l'addition des deux mots 0110 0100 (qui code +100) et 0010 0010 (qui code +34). Convertis le mot obtenu en valeur signée. Le

résultat est-il celui attendu ? Explique l'anomalie et propose un remède.

✔ Corrigé
Addition bit à bit : 0110 0100 + 0010 0010 = 1000 0110. Le bit de gauche du résultat vaut 1 : lu en complément à deux, ce mot vaut 134 − 256 = −122. Or 100 + 34 = +134 : le résultat obtenu (−122) est absurde, alors qu'aucune retenue ne déborde des 8 bits. C'est un dépassement de capacité (overflow) : +134 dépasse le maximum représentable sur 8 bits signés, qui est +127. Sa signature est reconnaissable : la somme de deux nombres positifs a produit un nombre négatif. Remède : effectuer le calcul sur davantage de bits — sur 9 bits, la plage signée va de −256 à +255 et +134 s'y code sans erreur.

Exercice 3

Un dictaphone numérique enregistre 30 secondes de son échantillonné à 22 050 Hz, en 16 bits, en stéréo. Calcule la taille brute de l'enregistrement en octets, puis en mégaoctets. Compare avec la qualité CD pour la même durée et explique d'où vient le rapport entre les deux tailles.

✔ Corrigé
Chaque échantillon occupe 16 bits, soit 2 octets, et il y a 2 canaux. Par seconde : 22 050 × 2 × 2 = 88 200 octets. Pour 30 secondes : 88 200 × 30 = 2 646 000 octets, soit environ 2,65 Mo. En qualité CD (44 100 Hz), le même enregistrement de 30 secondes pèserait 176 400 × 30 = 5 292 000 octets, soit environ 5,29 Mo : exactement le double. Le rapport vient de la fréquence d'échantillonnage, moitié moindre (22 050 = 44 100 ÷ 2), tous les autres paramètres étant identiques : la taille d'un son brut est proportionnelle à la fréquence choisie.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Le nombre décimal 25 s'écrit en binaire…

   A) 11001

   B) 10101

   C) 11100

2. Le nombre hexadécimal FF vaut en décimal…

   A) 256

   B) 255

   C) 240

3. Sur 8 bits en complément à deux, la plus petite valeur représentable est…

   A) −127

   B) −255

   C) −128

4. Le code ASCII décimal de la lettre « A » est…

A) 65

B) 97

C) 48

5. Avec 10 bits, on peut former combien de valeurs différentes ?

A) 1 000

B) 1 024

C) 512

6. En parité paire, le bit à ajouter au mot 1011 (trois bits à 1) est…

A) 0

B) 1

C) Cela dépend du mot suivant

7. Une image de 100 × 100 pixels en niveaux de gris sur 8 bits occupe, sans compression…

A) 10 000 octets

B) 80 000 octets

C) 800 octets

8. En UTF-8, le caractère « é » occupe…

A) 1 octet

B) Toujours 4 octets

C) 2 octets

9. En binaire, 1010 + 0101 vaut…

A) 1111

B) 10101

C) 0111

10. Le bit de parité permet de…

A) Corriger n'importe quelle erreur de transmission

B) Détecter une erreur touchant un seul bit, mais pas deux erreurs simultanées

C) Rendre la transmission plus rapide

Corrigés du QCM

1. Réponse A — 25 = 16 + 8 + 1, donc les bits de poids 16, 8 et 1 sont à 1 : 11001.

2. Réponse B — F vaut 15, donc FF = 15 × 16 + 15 = 240 + 15 = 255.

3. Réponse C — La plage signée sur n bits va de -2^{n-1} à $2^{n-1} - 1$; pour n = 8 : de −128 à +127.

4. Réponse A — « A » = 65 ; 97 est le code de « a » minuscule, et 48 celui du caractère « 0 ».

5. Réponse B — Chaque bit double le nombre de combinaisons : 2^{10} = 1 024 valeurs.

6. Réponse B — Le nombre de 1 est impair (3) : il faut ajouter un bit à 1 pour rendre le total pair (4).

7. Réponse A — 100 × 100 = 10 000 pixels, à 1 octet par pixel (8 bits) : 10 000 octets. Les 80 000 seraient des bits, pas des octets.

8. Réponse C — Les caractères ASCII tiennent sur 1 octet en UTF-8 ; les lettres accentuées comme « é » en demandent 2 ; seuls certains symboles et émojis montent à 3 ou 4 octets.

9. Réponse A — 1010 = 10 et 0101 = 5 ; 10 + 5 = 15, et 1111 = 8 + 4 + 2 + 1 = 15.

10. Réponse B — Il détecte les erreurs en nombre impair (dont l'erreur unique), ne localise rien, ne corrige rien, et deux erreurs simultanées se compensent.

Résumé — les notions importantes

• Une base b utilise b symboles ; la valeur d'un nombre est la somme de ses chiffres multipliés par les puissances de la base selon leur rang (base 10 : notre habitude, base 2 : la machine, bases 8 et 16 : des écritures compactes du binaire).

• Conversions : vers le décimal par la somme pondérée ; depuis le décimal par divisions successives, restes lus à l'envers ; entre bases 2, 8 et 16 par regroupements de 3 ou 4 bits, sans repasser par le décimal.

• Arithmétique binaire : la retenue arrive dès 2 (1 + 1 = 0, retenue 1) ; la soustraction emprunte 2 ou se ramène à une addition ; la multiplication additionne des produits partiels décalés. Toujours contrôler en décimal.

• Complément à deux : −x s'obtient en inversant les bits de x et en ajoutant 1 ; sur 8 bits, la plage signée va de −128 à +127, le bit de gauche donne le signe, et l'addition ordinaire fonctionne pour tous — à surveiller : le dépassement de capacité.

• Texte : ASCII associe à chaque caractère un numéro de 0 à 127 (« A » = 65, « 0 » = 48) ; Unicode couvre toutes les écritures du monde et UTF-8 stocke chaque caractère en 1 à 4 octets selon son point de code.

• Multimédia : taille d'image = pixels × octets par pixel (24 bits = 3 octets) ; taille de son = fréquence × octets par échantillon × canaux × durée. JPEG, PNG et MP3 existent parce que ces tailles brutes sont énormes.

• Fiabilité : le bit de parité rend les erreurs visibles en ajoutant une redondance calculée ; il détecte les erreurs en nombre impair, sans corriger, et les codes CRC ou de Hamming prolongent cette idée dans les systèmes réels.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Base 10, base 2, base 8 et base 16 : les conversions
Le poids d'un chiffre est la base élevée à son rang ; le rang se compte à partir de 0 en partant de la droite. ; Décimal vers base b : divisions successives par b ; les restes se lisent de bas en haut, le dernier reste est le chiffre de gauche. ; Binaire ↔ octal par paquets de 3 bits, binaire ↔ hexadécimal par paquets de 4 bits, toujours en regroupant depuis la droite.

Fiche 2 — Opérations arithmétiques en binaire
1 + 1 = 0, retenue 1 : toute l'arithmétique binaire repose sur cette ligne ; une retenue peut se propager sur plusieurs colonnes. ; Emprunter dans une soustraction ajoute 2 au chiffre (et non 10 comme en décimal) et diminue le chiffre voisin de 1. ; Toujours vérifier une opération binaire en reconvertissant opérandes et résultat en décimal : les deux calculs doivent raconter la même histoire.

Fiche 3 — Représentation des nombres négatifs : le complément à deux
Passer de x à −x, ou de −x à x : toujours la même recette — inverser tous les bits et ajouter 1. ; Sur n bits en complément à deux : minimum = -2^{n-1}, maximum = $2^{n-1} - 1$; sur 8 bits, de −128 à +127. ; Dépassement : si l'addition de deux nombres de même signe produit un résultat de signe opposé, le résultat est faux (exemple : 100 + 50 sur 8 bits signés).

Fiche 4 — Codage des caractères : ASCII et Unicode
Le caractère « 0 » (code 48) n'est pas le nombre 0 : un chiffre dans un texte est un caractère comme un autre, avec son propre code. ; ASCII ne connaît ni accents ni alphabets non latins : les erreurs d'encodage (caractères affichés de travers) viennent presque toujours d'une mauvaise lecture de la table ou de l'encodage. ; UTF-8 utilise une taille variable : 1 octet pour l'ASCII historique, 2 pour la plupart des lettres accentuées, jusqu'à 4 pour les émojis — d'où des tailles de fichiers texte qui varient selon le contenu.

Fiche 5 — Codage des images et du son : initiation
24 bits par pixel = 3 octets, un par composante rouge, verte, bleue, chacune codée sur 256 niveaux (0 à 255). ; Doubler la largeur ET la hauteur d'une image multiplie sa taille brute par 4 : la résolution croît comme une surface. ; Pour le son, retenir la formule complète avec les canaux : oublier la stéréo (× 2) est l'erreur la plus fréquente dans ces calculs.

Fiche 6 — Codes détecteurs d'erreurs : le bit de parité
En parité paire, le bit ajouté vaut 0 si le nombre de 1 des données est déjà pair, et 1 sinon ; en parité impaire, c'est l'inverse. ; Une erreur unique est toujours détectée ; deux erreurs simultanées se compensent et passent inaperçues : toujours énoncer cette limite. ; Détecter n'est pas corriger : le bit de parité ne localise pas l'erreur, le récepteur ne peut que rejeter le mot ou demander une retransmission.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Le Binaire pour Debutants - Cours Complet en Francais

Chaîne : lioncode · Langue : Français · Niveau : Débutant

Un cours complet et progressif sur le binaire : bits et puissances de 2, conversions dans les deux sens, octets et unités, addition et soustraction binaires, puis l'hexadécimal comme raccourci du binaire. Il couvre les leçons 1 et 2 et se termine par un récapitulatif à garder sous la main.

https://www.youtube.com/watch?v=TCuz0pR8Si0

Leçon 1 — Le codage - Le décodage - Le transcodage (Les systèmes de numération)

Chaîne : Technologie en poche · Langue : Français · Niveau : Débutant

Codage, décodage, transcodage : la vidéo enchaîne les conversions décimal vers binaire et hexadécimal, le retour vers le décimal, et la conversion directe entre binaire et hexadécimal. Un tableau d'équivalence entre bases sert de fil conducteur, parfait en complément de la leçon 1.

https://www.youtube.com/watch?v=m-5ot0WWg5I

Leçon 2 — Addition & Soustraction en Binaire, Octal, Décimal, Hexa | Exercices Corrigés | LCS

Chaîne : Dr Telecom · Langue : Français · Niveau : Débutant

Des additions et soustractions corrigées étape par étape, en binaire mais aussi en octal et en hexadécimal. À regarder après la leçon 2, exercice personnel à la main avant de dévoiler chaque correction.

https://www.youtube.com/watch?v=IV7zSewaUFs

Leçon 3 — Entiers négatifs (complément à 2)

Chaîne : Vretrocomputing · Langue : Français · Niveau : Débutant à intermédiaire

Comment les entiers négatifs sont représentés en binaire : du complément décimal à l'idée générale jusqu'au complément à deux, avec ses limites de plage et les opérations. Une visualisation utile de la leçon 3, notamment pour le dépassement de capacité.

https://www.youtube.com/watch?v=WX3XLJBSADk

Leçon 4 — Comment un ordinateur stocke les lettres ? (ASCII expliqué simplement)

Chaîne : Decode Dev · Langue : Français · Niveau : Débutant

Chaque lettre reçoit un numéro dans la table ASCII — A vaut 65 — et ce numéro est converti en binaire comme n'importe quel entier, avant le décodage inverse à l'affichage. La vidéo annonce aussi la suite : Unicode, les accents et les émojis, au cœur de la leçon 4.

https://www.youtube.com/watch?v=U8y3Wx2zpuE

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Tanenbaum, Andrew — « Architecture de l'ordinateur » (Pearson) : les chapitres consacrés à la représentation des nombres, des caractères et des données multimédias.

• The Unicode Consortium — documentation officielle (unicode.org) : tables de points de code et description de l'encodage UTF-8.

• Documentation Python (docs.python.org/fr) : fonctions bin, oct, hex, ord et chr pour refaire et vérifier tous les calculs du cours.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.